



Technical Reference

BrokerVerse OOTB: code

Prepared by – **iorta TechNXT**

Version – **1.0**

Date – **03 October 2026**

TABLE OF CONTENTS

1. Document Control.....	4
2. Acronyms.....	4
3. Introduction.....	5
3.1 Purpose and readers.....	5
3.2 Sources and how the figures were measured.....	5
3.3 Companion workbook.....	6
4. Solution layout.....	7
4.1 Repository structure.....	7
4.2 Back-end start-up and request pipeline.....	7
4.3 Back-end module pattern.....	8
4.4 Shared libraries.....	9
4.5 Database, migrations and seeds.....	9
4.6 Scheduled jobs.....	10
4.7 Front-end structure.....	10
4.8 How a request flows from a screen to the database and back.....	11
5. Coding standards followed in this code base.....	12
5.1 Naming and file layout.....	12
5.2 Routes are declared through the registry.....	12
5.3 Validation with zod.....	12
5.4 Errors and the response envelope.....	13
5.5 SQL: parameters, pool and transactions.....	13
5.6 Audit trail.....	13
5.7 Configuration through app_settings.....	14
5.8 Secrets and environment.....	14
5.9 Logging.....	14
5.10 Front-end conventions.....	14
6. Code quality controls.....	16
6.1 Size of the code base.....	16
6.2 Static checks.....	16
6.3 Automated tests.....	16
6.4 Consistency checks.....	17
6.5 Migration discipline.....	17
6.6 UAT scenario script.....	17
6.7 Continuous integration.....	17
7. Security controls in code.....	18
7.1 Authentication.....	18

7.2 Password policy, lockout and two-factor sign-in.....	18
7.3 Authorisation.....	18
7.4 SQL injection prevention.....	19
7.5 Input validation and request limits.....	20
7.6 HTTP headers, CORS and rate limiting.....	20
7.7 File uploads and downloads.....	20
7.8 Encryption and secrets.....	21
7.9 Audit and sign-in logging.....	21
7.10 OWASP Top 10 (2021) mapping.....	21
7.11 Points for the security review before go-live.....	22
8. Front end to back end dependencies.....	23
8.1 How the matrix was built.....	23
8.2 Summary per menu area.....	23
8.3 Prerequisites per back-end module.....	24
8.4 Set-up order used by the UAT scenario.....	25
8.5 Before enhancing a module.....	25
9. API catalogue summary.....	26
9.1 Conventions.....	26
9.2 Endpoints per module.....	26
9.3 Public endpoints.....	27
9.4 Keeping the API documentation current.....	27
10. Guide for the production support team.....	29
10.1 Running locally.....	29
10.2 Where the logs are.....	29
10.3 Tracing a defect from a screen to the SQL.....	29
10.4 Common failure patterns and where to look.....	30
10.5 Making a change safely.....	30
10.6 Release and rollback.....	31
10.7 Coding checklist for a change.....	32
11. Document Sign-Off.....	33

1. Document Control

Date	Version	Author & Business Unit Info	Reviewed By & Business Unit Info	Approved By & Business Unit Info	Change History
03 October 2026	1.0	iorta TechNXT			Initial issue

2. Acronyms

Acronym / Symbol	Definition / Meaning
OOTB	Out of the box
API	Application programming interface
JWT	JSON Web Token
SQL	Structured Query Language
ESM	ECMAScript modules
CRA	Create React App
GL	General ledger
LGU	Local government unit
BIR	Bureau of Internal Revenue
IC	Insurance Commission
TOTP	Time-based one-time password
HMAC	Hash-based message authentication code
CORS	Cross-origin resource sharing
OWASP	Open Worldwide Application Security Project
UAT	User acceptance testing
CI	Continuous integration
PM2	Node.js process manager used on the EC2 server

3. Introduction

3.1 Purpose and readers

This document describes how BrokerVerse OOTB is built: where the code is, the conventions the code follows, the controls that keep it correct and secure, which screen depends on which API, and how the production support team traces and changes it. It is written for the developers who enhance the product and for the support team that takes it over after go-live.

The business use of each screen is in the User Manual. Deployment steps are in `deploy/README.md` and `deploy/REFERENCE.md` of the repository. This document points to them where they apply and does not repeat them.

3.2 Sources and how the figures were measured

Every statement in this document comes from the repository as it stood on 03 October 2026 (branch with the commit "Add/edit forms as side panels, return to the list after saving receipts, vouchers and petty cash"). The figures were measured as follows.

Figure	How it was measured
Endpoints, permissions, screens	Route registry loaded by <code>backend/src/tools/export-api.js</code> (the same loader as <code>npm run export:api</code>)
Lines of code	Line count of <code>.js</code> , <code>.jsx</code> , <code>.sql</code> and <code>.scss</code> files, <code>node_modules</code> excluded
Tables, settings, roles, permissions	A new database migrated and seeded with reference data only (<code>SEED_SAMPLE_DATA=false</code>)
Back-end tests	<code>npm test (vitest)</code> on 03 October 2026: 65 test files, 634 tests, all passing
Front-end tests	<code>CI=true npx craco test --watchAll=false</code> : 11 suites, 43 tests, all passing
UAT scenario	<code>backend/scripts/uat-scenario.js</code> on a new database on 03 October 2026: 371 steps, 0 failures
Screen dependencies	Menu tree, routes and the import closure of every screen, matched against the route registry (section 8.1)

3.3 Companion workbook

The full reference lists are in the workbook

BrokerVerse_API_and_Dependency_Catalogue.xlsx, delivered with this document:

- **APIs:** every endpoint with its back-end module, method, path, summary, permission and screen (854 rows).
- **Screen dependencies:** every menu screen with its route, front-end file, service files, the endpoints it calls, the back-end modules behind them and the data that must exist first (172 rows).
- **Summary:** endpoints per back-end module by method, the number of menu screens that use each module, and the prerequisites per module.

4. Solution layout

4.1 Repository structure

BrokerVerse is one repository with two applications and their deployment files.

Folder	Content
backend/	REST API and scheduled jobs: Node.js 22, ECMAScript modules, Express 4, PostgreSQL through pg
backend/src/modules/	44 business modules, one folder each (router, service, SQL)
backend/src/lib/	Shared libraries: authentication, errors, validation, settings, audit, logging, PDF, uploads
backend/src/db/	Connection pool, migration runner, seed runner, 95 migrations, reference and sample seeds
backend/src/jobs/	Cron scheduler, job handlers, housekeeping
backend/test/	vitest suite (65 test files) with helpers and fixtures
backend/scripts/	Settings check, UAT scenario, sample-data purge, database creation, upload templates
backend/docs/	API documentation output (OpenAPI, Postman, Excel touchpoints) and the module guide
brokerverse/	Web application: React 18 built with Create React App and craco
brokerverse/src/	Screens (module/, agentModule/), services, routes, menu, theme, translations
deploy/	Runtime reference, EC2 deployment script, PM2 and nginx configuration, environment examples
docker-compose.yml	PostgreSQL 16, API and web on one server (test system or trial)
.github/workflows/	CI for pull requests (ci.yml) and the front-end deployment (deploy-frontend.yml)

Main run-time libraries of the back end: express 4.21, pg 8.13, zod 3.24, jsonwebtoken 9, bcryptjs 2.4, helmet 8, cors 2.8, multer 1.4, pino 9 with pino-http 10, node-cron 4, nodemailer. Main libraries of the front end: react 18.2, react-router-dom 6.30, primereact 10.3 with primeflex, @reduxjs/toolkit 2, axios 1.20, formik 2, i18next 25 with react-i18next, chart.js 4, moment.

4.2 Back-end start-up and request pipeline

backend/src/server.js starts the API in this order:

1. Refuse to start in production with an unsafe configuration (assertProductionConfig() in src/config.js).
2. Apply pending migrations (src/db/migrate.js), one instance at a time under a PostgreSQL advisory lock.
3. Run the idempotent seed (src/db/seed.js): roles, permissions, administrator, settings, scheduled jobs, reference data.
4. Build the Express application (createApp() in src/app.js) and listen on PORT.
5. Start the cron scheduler and the watch for changed schedules, then report ready on GET /api/health.

createApp() sets up the middleware in a fixed order. Every request passes through it before it reaches a module.

Order	Middleware	Source	What it does
1	requestId	src/app.js	Takes a well-formed x-request-id or creates a UUID; echoes it on the

Order	Middleware	Source	What it does
			response
2	requestContext	src/lib/requestContext.js	Keeps the request in AsyncLocalStorage (current user for document builders)
3	helmet	helmet	Security headers
4	cors	cors	Origins from CORS_ORIGINS; exposes x-request-id, Retry-After, Content-Disposition
5	pinoHttp	src/lib/logger.js	One JSON log line per request with the request id; secrets redacted
6	apiRateLimit	src/lib/rateLimit.js	Per user (valid token) or per IP; setting security.api_rate_limit
7	express.json	express	Body limit JSON_BODY_LIMIT (2mb); keeps the raw body for payment webhooks
8	signFileLinks	src/lib/fileLinks.js	Signs stored-file links in responses
9	Module routers	src/modules/*/router.js	Mounted under /api by loadModules()
10	Not-found and errorHandler	src/lib/errors.js	One JSON error envelope for every failure

4.3 Back-end module pattern

Every folder under backend/src/modules that holds a router.js is mounted under /api automatically. A module exports its router as default, and may export mount (path prefix), extraMounts (further routers at their own prefix) and order. The files of a module follow one pattern.

File	Role
router.js	Declares every endpoint with define(): method, path, summary, screen, permission middleware, zod validation, example request and response, handler
service.js	Business rules and SQL; receives a database client (db) when it must run inside a transaction
Named files (billing.js, placements.js, fiscal.js ...)	Larger modules split their service by topic
README.md (some modules)	Module notes: accounting, period-end, reports, document numbering

A route declaration from the receipts module (backend/src/modules/receipts/router.js, lines 141 to 151):

```
define({
  method: 'POST', path: '/', summary: 'Finance (write:receipts): create an official receipt ...',
  screen: `${SCREEN} > Add receipt`, middleware: [...write, validate(receiptSchema)],
  request: { receiptType: 'Payment', receiptDate: '2026-09-28T00:00:00.000Z', ... },
  response: { success: true, data: example },
  handler: async (req, res) => {
    const r = await withTransaction((db) => svc.createReceipt(db, req.body, req.user));
    await audit(req, { entity: 'receipt', entityId: r.receiptId, action: 'create', after: r });
    created(res, r, `Receipt ${r.receiptNumber} created`);
  },
});
```

write is [requireAuth, requirePermission('write:receipts')], declared once at the top of the router. The handler holds no SQL: the service does the work inside one transaction, the router writes the audit record and the response.

4.4 Shared libraries

File in backend/src/lib	Used for
registry.js	moduleRouter() and define(): one declaration feeds Express, the API list, OpenAPI and Postman
auth.js	Tokens, requireAuth, requirePermission, requireRole, isAdmin, session revocation
errors.js	HttpError, badRequest, unauthorized, forbidden, notFound, conflict, errorHandler
validate.js	validate(schema, part) middleware over zod
respond.js	ok(), created(), paging(), pageMeta(), wrap()
settings.js	getSetting(), getSettings(), setSetting() over app_settings, with a cache
audit.js	audit(req, { entity, entityId, action, before, after }) into audit_log
scope.js	Record-level scoping of users whose roles are all in security.scoped_roles
makerChecker.js	Approver must differ from the maker (finance.maker_checker_enabled)
password.js, totp.js, secrets.js	Password policy, two-factor codes, encryption at rest, signed links
rateLimit.js	Sign-in limiter and the global API limiter
uploadLimits.js	Multer limits, ZIP inflation cap, row cap for imports
numbering.js	nextDocumentNumber() over the document_numbering series
logger.js, loginHistory.js, health.js	Logging with redaction, sign-in history, readiness and liveness
pdf/, letterhead.js, template.js	PDF documents with the company letterhead; template variables
xlsx.js, csv.js, zip.js	Spreadsheet and CSV output and reading of uploaded workbooks
money.js, dates.js, birthDate.js	Rounding, business dates in the configured time zone
mailer.js, publicWeb.js	E-mail through SMTP_URL; public web address for links

4.5 Database, migrations and seeds

The database is PostgreSQL 16 (the version in docker-compose.yml and in CI). A database migrated and seeded on 03 October 2026 holds 167 tables, 59 functions, 404 settings, 7 roles, 64 permissions, 53 master types, 39 report definitions and 16 scheduled jobs.

- **Migrations** (backend/src/db/migrations, 95 files from 0001_core.sql to 0242_product_template_lob_codes.sql) run in file-name order. Each runs in its own transaction and is recorded in schema_migrations. Applied migrations are never edited; a correction is a new file.
- **Seeds** (backend/src/db/seeds) run on every start and are idempotent: rows are inserted by natural key and existing rows and administrator edits are kept. settings.json holds configuration keys, jobs.json the scheduled jobs,

the numbered SQL files the reference data (masters, chart of accounts, motor tariff, product templates, report catalogue, security). seeds/sample/ holds demo data, loaded only when SEED_SAMPLE_DATA is on.

- **Connection pool** (backend/src/db/pool.js): one pg pool of 10 connections; query(), one(), many() and withTransaction(fn). NUMERIC and BIGINT values come back as numbers and DATE values as YYYY-MM-DD strings.

4.6 Scheduled jobs

backend/src/jobs/scheduler.js reads scheduled_jobs and runs each enabled job with node-cron in the business time zone (general.timezone). A run takes a PostgreSQL advisory lock per job code, so a job runs once even when several API instances run the scheduler; SCHEDULER_ENABLED=false keeps an instance out of scheduling. Every run is a row of job_runs. The seeded jobs are listed below; administrators change the timing on Master > Schedules.

Job code	Cron (business time zone)
email-outbox	every 5 minutes
renewal-queue	every minute
policy-expiry, quote-expiry	00:15, 00:30 daily
recurring-journals, dormant-users, period-auto-soft-close, housekeeping	01:15, 01:45, 02:00, 02:45 daily
daily-reports, bank-auto-match, renewal-pipeline	05:00, 05:45, 05:30 daily
renewal-notice, receivable-ageing, collection-reminders, month-end-reminder	06:00, 07:00, 08:00, 08:00 daily
accrual-reversal	00:30 on the first day of the month

4.7 Front-end structure

Path in brokerverse/src	Content
index.js, App.js	Entry point; store, router, theme, session renewal; sign-in and public routes (/login, /approve-quote)
routes/MainRoute.js	Every signed-in route (439 Route elements), wrapped by routes/ProtectedRoute
components/SideBar/list.js	The side menu as one tree; 172 menu screens
utils/menuPermissions.js	roleMenuPermissions: which role sees which menu entry; route guard isPathAllowed
utils/canOpen.js	canOpen(path) and hasPermission(code) to hide links and actions
module/	Back-office screens: accounts, finance, masters, placement, remittance, reinsurance, commission, incentive, product configurator, reports, system settings
agentModule/	Operations screens: leads, clients, quotations, policies, endorsements, claims, renewals, payments, open items
services/	49 files, one per API area; all HTTP calls of the screens go through them
utility/	API client (interceptor.js, commonServices.js), token handling, session refresh, idle sign-out, formatting
redux/	Store and reducers (Redux Toolkit)
locales/	en.json (7,523 keys) and th.json
theme/bdoi/	PrimeReact theme and application styles (tokens.scss, bdoi.scss, enterprise.scss)

The front end reaches the API in two ways. Most services call fetch with the bearer header from authService.getAuthHeader(); a few use the axios instance in

utility/interceptor.js through getRequest, postRequest, putRequest, patchRequest and deleteRequest. Both paths renew an expired access token once: axios through its response interceptor, fetch through the wrapper installed by utility/sessionRefresh.js. The API base address comes from REACT_APP_BASE_URL at build time (utility/constant.js).

4.8 How a request flows from a screen to the database and back

The Add receipt screen saving a receipt shows the full path.

1. The user opens Accounts > Receipts. list.js gives the menu entry and its route /accounts/receipts; ProtectedRoute checks the path against the user's roles with isPathAllowed (deny by default).
2. The screen component calls receiptsService in brokerverse/src/services/receiptsService.js, which sends POST {REACT_APP_BASE_URL}/receipts with the bearer token and JSON body.
3. Express assigns the request id, applies the security headers, CORS, logging and the API rate limit, and parses the JSON body.
4. The receipts router runs requireAuth (signature, token type, user active, token version), requirePermission('write:receipts') and validate(receiptSchema). A failure ends here with 401, 403 or 400.
5. The handler opens a transaction with withTransaction(); svc.createReceipt(db, ...) takes a receipt number from the numbering series, writes the receipt and its lines, applies the payment to the open bill and posts the journal through the posting rules. All statements use \$1, \$2 ... parameters. Any error rolls the whole transaction back.
6. After the commit the handler writes audit_log and answers 201 { success: true, message, data } with the x-request-id header.
7. Any thrown error reaches errorHandler: a 4xx keeps its message, a 5xx answers a generic message with the request id and the detail goes to the log.
8. The screen shows the message and returns to the list.

5. Coding standards followed in this code base

This chapter records the conventions the existing code follows, with real excerpts. New code is expected to follow the same rules; the coding checklist in section 10.7 repeats them as a list.

5.1 Naming and file layout

- Back end: one folder per module in kebab case (bank-reconciliation, insurer-reconciliation); files in camel case (bundleQuotes.js, periodEndQueries.js); functions in camel case; constants in upper case (HEADER_FIELDS, PAYMENT_MODES).
- Database: tables and columns in snake case (receipt_lines, created_by); API fields in camel case (receiptNumber). Services map between them (for example HEADER_FIELDS in receipts/service.js).
- Permissions are read:<module>, write:<module> and approve:<module> (64 codes). Settings are <group>.<name> (limits.quote_validity_days).
- Front end: React components in Pascal case under module/<Area>/ or agentModule/<area>; one service file per API area named <area>Service.js.

5.2 Routes are declared through the registry

No module calls router.get() directly. moduleRouter() in backend/src/lib/registry.js returns define(), which places requireAuth first in the middleware chain unless the route is declared auth: false, wraps the handler so that rejected promises reach the error handler, and records the route for the documentation:

```
const define = (r) => {
  const method = r.method.toLowerCase();
  const auth = r.auth !== false;
  const given = (r.middleware || []).flat();
  const mws = auth ? [requireAuth, ...given.filter((m) => m !== requireAuth)] : given;
  router[method](r.path, ...mws, wrap(r.handler));
  ...
  ROUTES.push(entry);
};
```

A route therefore cannot be left open by forgetting requireAuth, and the API catalogue cannot drift from the code.

5.3 Validation with zod

Request bodies and query strings are validated with zod schemas through validate(schema, part) (backend/src/lib/validate.js). The parsed value replaces the original, so the handler receives typed data. A failure answers 400 with "Validation failed" and one message per field ("amount is required").

```
export const validate = (schema, part = 'body') => (req, _res, next) => {
  const r = schema.safeParse(req[part]);
  if (!r.success) return next(r.error);
  req[part] = r.data;
  return next();
};
```

Business checks that need the database (an open period, an existing policy, the approver's authority) are made in the service and throw the errors below.

5.4 Errors and the response envelope

Services throw the helpers of `backend/src/lib/errors.js` (`badRequest`, `forbidden`, `notFound`, `conflict`, or `HttpError` with any status). `errorHandler` turns every failure into one envelope:

```
{ "success": false, "message": "Validation failed",
  "errors": [{ "path": "amount", "message": "amount is required" }],
  "requestId": "6d1c..." }
```

A server error (5xx) never returns its detail to the browser. The body reads "Internal server error; quote the request id when reporting it" and the message, stack and database error go to the log with the same request id. Successful answers use `ok()` and `created()` from `backend/src/lib/respond.js`: `{ success: true, message, data }`, with pagination on lists.

5.5 SQL: parameters, pool and transactions

All SQL is plain text in the services and goes through the pg pool with numbered parameters. Values are never pasted into SQL text. An update of a receipt header (`backend/src/modules/receipts/service.js`, from line 194) shows the usual pattern for partial updates: the column names come from a fixed map in the code, the values from parameters.

```
const sets = []; const p = [r.id];
for (const [k, col] of Object.entries(HEADER_FIELDS))
  if (b[k] !== undefined && b[k] !== null) { p.push(b[k]); sets.push(`${col} = $$
{p.length}`); }
p.push(user.id); sets.push(`updated_by = ${p.length}`, 'updated_at = now()');
await db.query(`UPDATE receipts SET ${sets.join(', ')} WHERE id = $1`, p);
```

Operations that write more than one table run in `withTransaction()` (`backend/src/db/pool.js`): `BEGIN`, the function, `COMMIT`, or `ROLLBACK` on any error. Services take the client (`db`) as their first argument so that callers can compose them in one transaction. Rows that are changed under concurrency are read with `FOR UPDATE` first (for example `SELECT * FROM receipts WHERE id = $1 OR receipt_number = $1 FOR UPDATE`). Section 7.4 lists how the few dynamic identifiers are controlled.

5.6 Audit trail

Every handler that changes data calls `audit()` (`backend/src/lib/audit.js`) after the change, with the entity, its id, the action and the before and after images:

```
await audit(req, { entity: 'receipt', entityId: r.receiptId, action: 'create', after:
r });
```

`audit_log` records the user id, user name, IP address and time. Administrators read it on Master > Audit Trail (GET `/api/settings/audit`, permission `read:audit`). Modules

with a business history of their own keep it in addition: `claim_history`, `period_status_history`, `bank_reconciliation_history`, `login_history`.

5.7 Configuration through `app_settings`

Business parameters are not written into the code. They are rows of `app_settings` (404 keys after seeding), read with `getSetting(key, fallback)` and edited on Master > Configuration and Master > System Settings. For example (backend/src/modules/quotations/service.js, line 89):

```
const validity = Number(await getSetting('limits.quote_validity_days', 30));
```

Settings are cached per API instance; the cache is checked against the table at most every 5 seconds (`SETTINGS_CHECK_MS`) and every entry expires after 60 seconds (`SETTINGS_CACHE_TTL_MS`), so a change saved on one instance reaches the others without a restart. A new key is added in a seed or migration with `ON CONFLICT (key) DO NOTHING` and a label; `npm run check:settings` fails when the code reads a key that no migration or seed creates.

5.8 Secrets and environment

Deployment values and secrets come from the environment only (backend/src/config.js, backend/.env.example): `DATABASE_URL`, `JWT_SECRET`, `DATA_ENCRYPTION_KEY`, `CORS_ORIGINS`, `PUBLIC_BASE_URL`, `SMTP_URL`, upload limits and token lifetimes. Payment gateway credentials are read from `<prefix>_SECRET_KEY` and `<prefix>_WEBHOOK_SECRET` variables (backend/src/modules/payment-gateway/providers.js), never from the database. The development fallbacks in config.js are refused in production: with `NODE_ENV=production` the server does not start when `JWT_SECRET` or `DATA_ENCRYPTION_KEY` is missing, a placeholder, shorter than 32 characters or equal to the other, when `CORS_ORIGINS` is `*`, or when `PUBLIC_BASE_URL` is missing or a localhost address.

5.9 Logging

The back end logs JSON lines to standard output with pino (backend/src/lib/logger.js). `console.log` is an ESLint error in application code. Each request line carries the request id; job lines carry the job code and run id. Authorization headers, cookies, passwords, refresh tokens, one-time codes and signed-link parameters are redacted (`REDACT_PATHS` and `redactUrl`). The log level comes from `LOG_LEVEL`. In the front end, `utility/logger.js` replaces direct `console` calls (no-console is a warning there).

5.10 Front-end conventions

- Every HTTP call goes through a file in `services/`; screens do not build URLs themselves.
- Screen texts come from `locales/en.json` through `useTranslation()` and `t("key");` `npm run check:i18n` lists keys used in code and missing from `en.json`.

- Menu visibility and route access come from `roleMenuPermissions` (deny by default). Buttons for actions the API would refuse are hidden with `hasPermission("write:...")`; the server stays the authority.
- Colours, spacing and component styles come from the BDOI theme tokens in `theme/bdoi/`; `npm run theme:bdoi` rebuilds the theme.

6. Code quality controls

6.1 Size of the code base

Area	Files	Lines
Back end: modules (src/modules)	169	34,207
Back end: shared libraries (src/lib)	39	2,919
Back end: migrations	95	5,324
Back end: seeds (reference and sample)	35	3,555
Back end: jobs, tools, server, configuration, database runners	10	1,069
Back end: tests (test/)	74	13,455
Back end: scripts (settings check, UAT scenario, purge, templates)	28	3,158
Front end: module/	575	138,962
Front end: agentModule/	351	81,730
Front end: services/	50	10,650
Front end: components, routes, utilities, store, hooks, context	80	11,211
Front end: theme and styles	5	8,266
Front end: translations (en.json, th.json)	2	13,373

Inventory item	Count
Back-end module folders / module labels in the API	45 / 55
Endpoints in the route registry (plus 2 health endpoints)	868
Menu screens / route elements in MainRoute.js	170 / 439
Front-end service files / API call sites found by the checker	49 / 767
Database tables / functions after migration	167 / 59
Settings / permissions / roles / master types / report definitions	404 / 64 / 7 / 53 / 39

6.2 Static checks

- **Back end:** ESLint 9 with backend/eslint.config.js: the recommended rule set plus no-console (error), no-unused-vars (error, arguments starting with _ allowed), eqeqeq (smart) and prefer-const. Command-line scripts may write to the console. npm run lint on 03 October 2026: no errors, no warnings.
- **Front end:** the Create React App rule set (react-app, react-app/jest in brokerverse/package.json) with no-console as a warning. A run over src on 03 October 2026: 898 files, no errors, 299 warnings (161 react-hooks/exhaustive-deps, 121 eqeqeq, 17 others). Warnings do not stop the build.

6.3 Automated tests

- **Back end:** vitest, 65 test files with 634 tests, run against a real PostgreSQL database (TEST_DATABASE_URL, default brokerverse_test) with fileParallelism: false. The suite covers every business module through the HTTP API with supertest (for example receipts.test.js, period-end.test.js, remittance.test.js), the security controls (security.test.js, hardening.test.js, role-access.test.js, scope.test.js), configuration (configuration.test.js, configuration-controls.test.js) and the tools (tools.test.js). Result on 03 October 2026: all 634 tests passed.
- **Front end:** jest through craco test, 11 suites with 43 tests: menu and route permissions (utils/menuPermissions.test.js, utils/canOpen.test.js), number

and date formatting, configured options, form error helpers, and the lead and masters services. Result on 03 October 2026: all passed.

6.4 Consistency checks

Command	What it checks	Result on 03 October 2026
<code>npm run check:settings</code> (back end)	Every key read in code exists in a migration or seed; lists unread keys	0 missing; 1 unread (<code>product.component_kinds</code>); 5 read only by the front end
<code>npm run check:api</code> (front end)	Every front-end call matches a back-end route and method	0 calls to missing routes against the current routes; 2 method warnings caused by a helper that chooses GET or POST at run time
<code>npm run check:i18n</code> (front end)	Translation keys used in code and missing from <code>en.json</code>	2 keys without an English text; 1,861 keys not yet in <code>th.json</code>
<code>npm run export:api</code> (back end)	Loads every module and writes OpenAPI, Postman and the touchpoint workbook	868 routes, no module skipped

`check:api` compares against `backend/docs/api/openapi.json`. Run `npm run export:api` in `backend/` first, so that the check uses the current routes.

6.5 Migration discipline

- One new file per change, numbered after the last one; applied files are never edited.
- Each file runs in its own transaction; a failure stops the start-up with the file name in the log and leaves the database at the previous file.
- Migrations only add or convert: new tables and columns, `IF NOT EXISTS`, `ON CONFLICT DO NOTHING`, updates with a `WHERE` clause.
- The runner takes a PostgreSQL advisory lock, so several API instances starting together apply the files once.
- `GET /api/health` answers 503 while migrations are pending; `GET /api/version` lists them.

6.6 UAT scenario script

`backend/scripts/uat-scenario.js` runs the full broking cycle of a Philippine non-life broker through the public HTTP API, with synthetic data spread over six months: personas, masters, sales, placement, billing, servicing, money, reconciliation, month-end and reports. The phases are in `backend/scripts/uat/phases/`. It is pointed at any environment with `API_BASE`, `ADMIN_PASSWORD` and `PERSONA_PASSWORD`, uses a fixed random seed so that a rerun gives the same data, and exits with code 1 when a step fails. Run on a new database on 03 October 2026: 371 steps, 0 failures.

6.7 Continuous integration

`.github/workflows/ci.yml` runs on every pull request. The back-end job starts PostgreSQL 16, installs with `npm ci`, runs `npm run lint` and `npm test`. The front-end job installs with `npm ci --legacy-peer-deps` and runs the jest suite. `deploy-frontent.yml` builds and publishes the front end. The back end on EC2 is installed by `deploy/ec2/deploy.sh` and run by PM2 (`deploy/ec2/ecosystem.config.cjs`).

7. Security controls in code

7.1 Authentication

- **Tokens.** Sign-in (POST /api/auth/login) returns a short access token (JWT, HS256 only, JWT_ACCESS_TTL_SECONDS, default 1,800 seconds) and a refresh token (default 30 days). Refresh tokens are stored in refresh_tokens by id, rotate on every use, and a reused refresh token revokes its whole family.
- **Every request** is checked by authenticate() in backend/src/lib/auth.js: signature and algorithm, token type access, the user still exists and is active, and the token version matches users.token_version. Raising the token version (password change or reset, deactivation, role change, Sign out everywhere) ends every session of that user at once.
- **Restricted sessions.** A user who must change the password or enrol in two-factor sign-in receives a restricted token that only reaches the change-password or enrolment endpoints.
- **Passwords** are hashed with bcrypt (cost 10). An unknown user name still costs one bcrypt comparison, so response time does not reveal which names exist. Password reset codes are stored as keyed hashes, expire after security.reset_code_minutes (15) and are withdrawn after security.reset_code_max_attempts (5) wrong entries.
- **Idle sign-out** in the browser after limits.session_idle_minutes (default 30) with a one-minute warning (brokerverse/src/utility/idleTimeout.js).

7.2 Password policy, lockout and two-factor sign-in

Control	Setting	Seeded value
Minimum length	security.password_min_length	8
Upper case, lower case, digit, symbol	security.password_require_*	all required
Password history	security.password_history_count	last 5 refused
Maximum age	security.password_max_age_days	90 days
Lockout after wrong passwords	limits.max_login_attempts	5 (status becomes locked)
Sign-in attempts per IP and per user name	security.login_rate_limit	10 per 300 seconds
Roles that must use two-factor sign-in	security.require_2fa_roles	none

Two-factor sign-in uses authenticator codes (TOTP, RFC 6238, backend/src/lib/totp.js). The secret is stored encrypted, a code cannot be used twice (totp_last_step), and wrong codes count against the sign-in rate limit. Administrators reset a user's two-factor enrolment with POST /api/users/:id/2fa/reset.

7.3 Authorisation

- **Permission on every route.** Of the 868 endpoints, 807 require a permission or role through requirePermission() or requireRole(), 42 require only a signed-in user (own profile, own notifications, own calendar events, address look-ups, file upload and download, global search, permission list for the role

screen), and 19 are public by design (section 9.3). A missing permission answers 403 naming the permission.

- **Roles.** Seven roles are seeded: System Administrator (system-admin, every permission), Sales & Marketing, Processing Team, Operations, Claims, Accounting and Accounting Manager. Roles can inherit other roles (user_effective_roles()); administrators change grants on Master > Generals > User Management.
- **Record scoping.** Users whose roles are all listed in security.scoped_roles see only their own book: lists add an ownership predicate (scopeSql()), and detail routes answer 404 for another user's record (ownRecord()), so the record's existence is not disclosed. The scope travels under a JavaScript Symbol, so it cannot be set from the query string.
- **Approvals.** The maker-checker rule (backend/src/lib/makerChecker.js) refuses an approval by the user who created the record; the authority matrix (assertAuthority() in access-control/service.js) checks the approver's limit per transaction type, raised by active delegations; segregation-of-duties rules are kept under Master > Generals > User Management.
- **Front end.** roleMenuPermissions in brokerverse/src/utils/menuPermissions.js denies by default and lists, per role, the menu entries it may open; ProtectedRoute applies the same rule to typed URLs. This only hides screens; the server enforces the permissions.

7.4 SQL injection prevention

Method. Every JavaScript file under backend/src was searched on 03 October 2026 for (a) SQL text built with + concatenation, (b) template literals that contain SQL keywords and interpolate a value (\${...}), and (c) interpolations that reference req.query, req.body or req.params directly. Each interpolated expression found in (b) was traced to its source.

Result.

- No SQL is built by string concatenation.
- No request value is interpolated into SQL text. The hits for (c) are messages and audit action names, not SQL.
- 953 SQL template literals were found; 378 of them interpolate an expression. Every interpolated expression is one of: a placeholder number (\${params.length + 1}), a WHERE clause assembled from fixed fragments whose values are parameters, a constant SQL fragment defined in the same file, or an identifier chosen from a fixed list in the code. All values go to pg as parameters.

The dynamic identifiers and how each is controlled:

Where	What is interpolated	Control
Sorting in lists (renewals/batches.js line 96, collections/service.js, claims/service.js)	ORDER BY column and direction	Column looked up in a fixed map; direction is ASC or DESC only
Partial updates (receipts, disbursements, claims, renewals, document	Column names in SET	Taken from fixed field maps in the service, never from the

Where	What is interpolated	Control
numbering, product configurator)		request
Generic inserts (clients, leads, quotations, placements, package quotes)	Column list	Keys of an object built in code from a fixed field map
Masters (masters/service.js)	Table and column names of table-backed masters	Table must be in the TABLES allow-list; identifiers pass IDENT (<code>^[a-z_][a-z0-9_]*\$</code>) and are double-quoted by <code>q()</code> ; field names of new master types are checked when the type is saved
Report engine (reports/engine.js)	Report query, columns, group-by	Queries come from QUERIES in code; column keys pass IDENT and are quoted by <code>q()</code> ; filters come from the fixed FILTERS map
Petty cash (payments/pettycash.js)	Table and number column	Looked up in fixed maps by entry kind
Document numbering (lib/numbering.js)	Table and column for the uniqueness check	Passed by code only and checked against IDENT
Housekeeping (jobs/housekeeping.js)	Table name and condition	Fixed list in the job

7.5 Input validation and request limits

- Bodies and query strings are validated with zod (section 5.3); services check business rules against the database.
- JSON bodies are limited to `JSON_BODY_LIMIT` (2mb). Uploads are held in memory and limited to `UPLOAD_MAX_MB` (10 MB) per file and `UPLOAD_MAX_FILES` (10) per request; imports to one file of `IMPORT_MAX_MB` (10 MB), `IMPORT_MAX_INFLATED_MB` (50 MB) once uncompressed, and `IMPORT_MAX_ROWS` (20,000) rows.
- Lists cap the page size at 500 rows (`paging()` in `respond.js`); report files cap at 50,000 rows.

7.6 HTTP headers, CORS and rate limiting

- helmet sets the security headers on every response; `x-powered-by` is off.
- CORS allows only the origins in `CORS_ORIGINS` (* is refused in production).
- The global limiter allows `security.api_rate_limit` requests per window (600 per 60 seconds) per signed-in user, or per IP without a valid token; the answer is 429 with `Retry-After`. The sign-in limiter is separate (section 7.2). Both count per API instance.
- `trust proxy` is set to one hop, so the client address is taken from the load balancer's header.

7.7 File uploads and downloads

- The type of an uploaded file is decided from its content signature and extension, never from the type the browser sends (`backend/src/modules/uploads/fileTypes.js`). Only types in

uploads.allowed_types are accepted: JPEG, PNG, GIF, WebP, PDF, CSV, plain text, Excel and Word.

- Stored files get a key with a 128-bit random part (storage.js), so keys cannot be guessed.
- Downloads need a bearer token or a signed link (?exp=&sig=, HMAC, valid FILE_URL_TTL_SECONDS, default 1,800 seconds). Files are served with X-Content-Type-Options: nosniff; everything except PDF gets a sandbox content security policy; only images and PDF open inline, other types download as attachments; HTML is never served as HTML.
- Replacing or deleting a file is limited to the uploader, a user with write permission on the module that owns it, or an administrator, and is audited.

7.8 Encryption and secrets

- Two-factor secrets are encrypted at rest with AES-256-GCM using a key derived from DATA_ENCRYPTION_KEY (backend/src/lib/secrets.js). One-time codes are kept as HMAC hashes and compared in constant time.
- DATA_ENCRYPTION_KEY must be kept with the database backups: without it the stored two-factor secrets cannot be read and users must enrol again.
- JWT_SECRET signs tokens and, through a derived key, file links. Changing it ends every session and every issued link.
- No secret is in the repository: .env files are local; examples carry placeholders that production refuses.

7.9 Audit and sign-in logging

- audit_log: every change with user, IP address, entity, action and before and after images (section 5.6).
- login_history: every sign-in attempt with result and reason (bad-password, unknown-user, locked, inactive, rate-limited, 2fa-required, bad-2fa-code).
- job_runs: every scheduled or manual job run with status, output and error.
- Request logs with request id and redaction (section 5.9).

7.10 OWASP Top 10 (2021) mapping

OWASP risk	Controls in BrokerVerse	Where in the code
A01 Broken access control	Permission on every route; deny-by-default menu; record scoping; maker-checker; authority matrix	lib/auth.js, lib/registry.js, lib/scope.js, lib/makerChecker.js, utils/menuPermissions.js
A02 Cryptographic failures	bcrypt passwords; AES-256-GCM for two-factor secrets; HMAC codes and links; HS256 only	lib/password.js, lib/secrets.js, lib/auth.js
A03 Injection	Parameterised SQL only; identifiers from allow-lists; zod validation	All services; section 7.4
A04 Insecure design	Restricted sessions; token versions; refresh rotation with reuse detection; segregation of duties	lib/auth.js, modules/auth/router.js, modules/access-control

OWASP risk	Controls in BrokerVerse	Where in the code
A05 Security misconfiguration	Production start-up check; helmet; CORS list; no stack traces to clients	config.js, app.js, lib/errors.js
A06 Vulnerable and outdated components	Locked dependency versions (package-lock.json); npm ci in CI	backend/package.json, brokerverse/package.json
A07 Identification and authentication failures	Password policy and history; lockout; sign-in rate limit; two-factor sign-in; idle sign-out	lib/password.js, lib/rateLimit.js, lib/totp.js, utility/idleTimeout.js
A08 Software and data integrity failures	Signed webhooks checked against the raw body; migrations in transactions; seeds idempotent	modules/payment-gateway/providers.js, db/migrate.js, db/seed.js
A09 Security logging and monitoring failures	Audit log; sign-in history; request ids; log redaction; job runs	lib/audit.js, lib/loginHistory.js, lib/logger.js
A10 Server-side request forgery	The API calls only fixed, configured addresses (payment gateways, SMTP); no request-supplied URL is fetched	modules/payment-gateway/providers.js, lib/mailer.js

7.11 Points for the security review before go-live

These are facts of the current design that the customer's security reviewer should confirm as acceptable.

- GET /api/settings returns every configuration value to any signed-in user. No secret is stored in settings, but security parameters (lockout, rate limits) are visible to all staff.
- A stored file can be read by any signed-in user who has its key. Keys are random and are only handed out in API responses to users who can see the record.
- Access and refresh tokens are kept in the browser's local storage.
- Rate-limit counters are kept in memory per API instance; with several instances behind a load balancer the effective limit is multiplied by the number of instances.
- security.require_2fa_roles is empty after seeding; the customer decides which roles must use two-factor sign-in.

8. Front end to back end dependencies

8.1 How the matrix was built

The matrix answers, for each menu screen, which front-end file, which service files and which endpoints it uses, which back-end module serves them, and which data must exist before the screen can be used. It was built from the code, not from memory:

1. The 172 menu screens were read from `brokerverse/src/components/SideBar/list.js` (path and the includes list of related pages).
2. Each path was resolved to its component through `brokerverse/src/routes/MainRoute.js`, and the component to its file.
3. The import closure of each component (and of the pages in its includes list) gave the service files and the service functions the screen uses.
4. The API call sites in those files were taken from the parser in `brokerverse/scripts/check-api-calls.js` and matched to the route registry. Endpoints whose registry screen label names the menu item were added.
5. The back-end module of each endpoint and the module prerequisites (section 8.3) complete the row.

The complete result, one row per menu screen, is the sheet **Screen dependencies** of the companion workbook. Where a screen uses a generic service (the masters service, the petty cash service), the row lists every endpoint of that service.

8.2 Summary per menu area

Menu area	Screens	Main service files	Back-end modules
Dashboard	4	dashboardService, reportsService	dashboard, claims, reports, policies
Operations (Home, Clients, Policy, Claims, Open Items, Payments)	6	policyService, paymentsService, clientService, claimsService, leadService	policies, payments, clients, claims, endorsements, documents
Operations > Sales & Marketing	6	placementService, quotationService, leadService, packagesService	leads, quotations, placement, packages, premium-charges, documents
Operations > Renewals	8	renewalsWorkspaceService, batchRenewalService	renewals
Accounts (Receipts, Collections, Disbursement, Journal Voucher, Correction and Reversal JV, Open Entry, Accounting Query)	10	receiptsService, collectionService, disbursementService, accountingService, journalVoucherService	receipts, collections, disbursements, accounting, journal-vouchers
Accounts > Credit Control	4	creditControlService	credit-control
Accounts > Remittance	16	remittanceService	remittance
Accounts > Petty Cash	5	pettyCashService	payments (petty cash)
Accounts > Bank Reconciliation	7	bankReconciliationService, reportsService	bank-reconciliation, reports
Accounts > Insurer Reconciliation	1	insurerReconciliationService	insurer-reconciliation

Menu area	Screens	Main service files	Back-end modules
Accounts > Tax	6	periodEndService, reportsService	period-end, reports
Accounts > Period End	5	periodEndService	period-end, accounting
Accounts > Incentive	4	incentiveService	incentive
Commission	2	commissionService	commission
Reinsurance	5	reinsuranceService	reinsurance
Reports (All Reports, Operational, Financial)	19	reportsService, periodEndService	reports, accounting
Master (System Settings, Configuration, Document Numbering, Schedules, Audit Trail, E-mail Outbox)	6	systemSettingsService, adminService, numberingService, emailService	settings, system-settings, document-numbering, schedules, notifications
Master > Generals	23	mastersService, accessControlService, userService	masters, access-control, users
Master > Finance	25	mastersService, postingRulesService, packagesService, periodEndService, remittanceService	masters, posting-rules, accounting, bank-reconciliation, packages, premium-charges, remittance, reinsurance
Product Configurator	10	productConfiguratorService, mastersService	product-configurator

8.3 Prerequisites per back-end module

A screen works only when the data its module reads exists. The table lists, per module, what must be in place first. The workbook repeats it on every screen row, together with the master types the screen reads.

Back-end module	Must exist first
quotations	Products and Insurance Company masters; coverages and vehicle masters for motor; motor tariff (seeded); Premium Taxes & LGU Rates (premium_charge_rules, lgu_tax_rates); Commission Rate Matrix; a prospect or client
placement	Client or prospect; Insurance Company and Products masters; numbering series for broker slips and placements
packages	Package Bundles; Insurer Rate Tables in force for the date; Premium Taxes & LGU Rates; Payment Gateways for payment links
policies	Accepted quotation or bound placement; insurer credit terms; posting rules and account roles for the booking journal; open accounting period
endorsements, claims	An issued policy; for claims the claims.* settings and signatories
renewals	Policies near expiry; renewals.* settings; jobs renewal-queue, renewal-pipeline and renewal-notices enabled
receipts	Open bills from issued policies, endorsements, renewals or go-live opening items; bank accounts; posting rules; open period
collections, credit-control	Open receivables; jobs receivable-ageing and collection-reminders; credit.* settings and client credit limits
disbursements	Payables (commission, refunds, premium due to insurers); bank accounts and checkbooks; posting rules; open period
remittance	Collected premiums due to insurers; Remittance Master (settlement limits)
commission, commission-rates	Insurance Company and Products masters; Commission Rate Matrix; referrers; issued policies with commission
reinsurance	Reinsurance Treaty master and reinsurers; policies and

Back-end module	Must exist first
	claims to cede
incentive	Incentive Programs master; issued policies; eligible roles setting
journal-vouchers, accounting	Chart of accounts; transaction codes; accounting periods
bank-reconciliation	Bank accounts linked to GL accounts; Bank Statement Formats and Bank Transaction Types; imported statements
insurer-reconciliation	Insurer Statement Formats; remittances to reconcile against
period-end	Fiscal years and periods; chart of accounts; tax codes; Close Checklist; posting rules
posting-rules	Active GL accounts; account roles under Account Determination
reports, dashboard	Report definitions (seeded); transactions in the source modules
documents	Signatories; company letterhead and logo (Company master, System Settings)
leads, clients	Numbering series; address masters (country, state, city)
notifications	SMTP_URL in the environment and notification.email_enabled for e-mail delivery

8.4 Set-up order used by the UAT scenario

The UAT scenario script loads a new database in the order below before it creates any business transaction (backend/scripts/uat/phases/setup.js). The same order is the safe order for a go-live load, because each step reads what the step before created.

1. Persona users with their roles.
2. Products master.
3. Insurers with credit terms.
4. Commission Rate Matrix.
5. Bank accounts linked to the ledger.
6. Insurer Statement Formats.
7. Tax codes of the commission taxes (checked).
8. Sub-agent referrers.
9. Settlement limit of the Remittance Master.

8.5 Before enhancing a module

1. Find the screen in the workbook (sheet Screen dependencies) and note its front-end file, service file, endpoints and back-end modules.
2. Read the module's router.js for the endpoints, their permission and validation schema, then the service functions they call.
3. Check the other screens that call the same endpoints (filter the Endpoints column): a change in the response shape affects all of them.
4. Check the prerequisites column: a new mandatory field may need data in a master or a new setting first.
5. Check the scheduled jobs and posting rules the module uses; a change in amounts reaches the ledger through posting rules.

9. API catalogue summary

9.1 Conventions

Topic	Convention
Base path	Every endpoint is under /api (for example https://<host>/api/receipts)
Authentication	Authorization: Bearer <access token> from POST /api/auth/login; renewed with POST /api/auth/refresh
Success response	{ "success": true, "message": "...", "data": ... }; 201 for created records
Lists	page and perPage (also pageNo, pageSize, limit); default 10 per page, at most 500; pagination: { total, page, perPage, totalPages }
Errors	{ "success": false, "message", "errors": [{ "path", "message" }], "requestId" }; 400 validation, 401 sign-in, 403 permission, 404 not found, 409 conflict, 413 too large, 415 file type, 429 rate limit, 500 server
Correlation	x-request-id header on every response; sent by the caller or created by the API
Files	PDF, XLSX and CSV downloads answer the file with Content-Disposition; stored files are reached through signed links
Identifiers	Most detail routes accept the database id or the document number (/receipts/OR-2026-00001)
Dates and money	Dates as YYYY-MM-DD in the business time zone; amounts as numbers in PHP rounded to 2 decimals

9.2 Endpoints per module

868 endpoints are registered: 397 GET, 341 POST, 87 PUT, 7 PATCH and 36 DELETE. The table shows the count per back-end module and the number of menu screens that call it. The workbook sheet **APIs** lists each endpoint with its summary, permission and screen.

Back-end module	Base path(s) under /api	Endpoints	Menu screens
access-control	/access-control	24	8
accounting	/accounting	27	63
addresses	/addresses	5	2
auth	/auth	15	1
bank-reconciliation	/bank-reconciliation	38	5
claims	/claims	17	4
clients	/clients, /customers	6	43
collections	/collections	9	1
commission	/commission	19	3
commission-rates	/commission-rates	7	1
credit-control	/credit-control	19	4
dashboard	/dashboard, /agent	6	5
disbursements	/disbursements	15	2
document-numbering	/document-numbering	5	1
documents	/document-templates	6	5
endorsements	/endorsements	8	2
incentive	/incentive	22	6
insurer-reconciliation	/insurer-reconciliation	18	2
journal-vouchers	/journal-vouchers	8	3
leads	/leads, /lead	11	11
masters	/masters	13	101
notifications	/notifications, /email	11	2

Back-end module	Base path(s) under /api	Endpoints	Menu screens
packages	/packages	27	3
payment-gateway	/payment-gateways, /payment-links, /public	13	1
payments	/payments, /open-items, /petty-cash	29	8
period-end	/period-end	45	8
placement	/broker-slips, /placements	26	10
policies	/policies	12	8
posting-rules	/posting-rules, /account-determination	22	4
privacy	/privacy	14	4
premium-charges	/premium-charges	9	6
product-configurator	/product-configurator	72	11
quotations	/quotations, /quote, /master, /email and coverage look-ups	42	4
receipts	/receipts, /billing-statement	19	3
reinsurance	/reinsurance	38	6
remittance	/remittance	86	17
renewals	/renewals, /policy-renewals	47	10
reports	/reports	13	31
schedules	/schedules	4	1
search	/search	1	0
settings	/settings	4	41
system	/version	1	0
system-settings	/system-settings	7	12
uploads	/s3, /upload	11	12
users	/users, /roles	17	7

The menu screen count of shared modules (masters, accounting, clients, settings, reports) is high because many screens read masters, settings and account lists or upload files. The two health endpoints (GET /api/health, GET /api/health/live) are mounted in src/app.js outside the registry.

9.3 Public endpoints

These endpoints answer without a bearer token. Each has its own control.

Endpoint	Control
POST /api/auth/login, /login/2fa, /refresh, /logout, /forgot-password, /reset-password; GET /api/auth/password-policy	Sign-in rate limit, lockout, hashed reset codes, refresh rotation
GET /api/s3/object/*, GET /api/upload/file/*	Bearer token or a valid signed link
GET /api/reports/generated/:id/download	Bearer token with report access, or a signed download token
POST /api/quotations/approve-by-customer	Signed approval token from the e-mail link
GET /api/public/payments/:token, /policy.pdf, POST /sandbox	Signed payment-link token; the simulation works only on sandbox payment links
GET, POST /api/public/payments/webhooks/:gateway	Gateway signature checked on the raw body
GET /api/settings/public, GET /api/system-settings, GET /api/version	Branding and version information only

9.4 Keeping the API documentation current

npm run export:api in backend/ loads every module and writes to backend/docs/api/:openapi.json (OpenAPI 3.1), the Postman collection and environment, BrokerVerse_API_Touchpoints.xlsx (API List, By Screen, Modules) and a CSV. The

Postman collection signs in first and keeps the token in `{{token}}`. Run it after every change to a router and commit the result with the change.

10. Guide for the production support team

10.1 Running locally

1. Install Node.js 22 and PostgreSQL 16; create the database brokerverse (or run `node scripts/create-database.js`).
2. In `backend/`: copy `.env.example` to `.env`, run `npm ci`, then `npm run dev`. The first start applies the migrations and the seed; the log prints the administrator password when `ADMIN_PASSWORD` is not set.
3. In `brokerverse/`: run `npm ci --legacy-peer-deps`, then `REACT_APP_BASE_URL=http://localhost:8000/api npm start (port 3000)`.
4. Sign in as BrokerVerse. With `SEED_SAMPLE_DATA` on (the development default) demo data is loaded.
5. Before a change: `npm run lint` and `npm test` in `backend/` (needs the test database `brokerverse_test`); `CI=true npm test -- --watchAll=false`, `npm run check:api` and `npm run check:i18n` in `brokerverse/`.

`docker compose up --build` at the repository root starts PostgreSQL, the API and the web application together on port 8080.

10.2 Where the logs are

Installation	API log	Other records
EC2 with PM2 (<code>deploy/ec2</code>)	<code>pm2 logs brokerverse-api</code> ; files under the PM2 home of root (<code>~/ .pm2/logs/</code>)	Database tables below
Docker Compose	<code>docker compose logs api</code>	Database tables below
Local development	The terminal running <code>npm run dev</code>	Database tables below

Database records that support the logs: `audit_log` (changes), `login_history` (sign-ins), `job_runs` (scheduled jobs), `email_outbox` (e-mails and their errors), `claim_history`, `period_status_history`, `bank_reconciliation_history`. Health: `GET /api/health` (readiness, database and pending migrations), `GET /api/health/live`, `GET /api/version`.

10.3 Tracing a defect from a screen to the SQL

1. **Get the request id.** The user quotes it from the error message, or the support analyst reads the `x-request-id` response header in the browser's network tab.
2. **Find the log line.** Search the API log for the request id. The line holds the method, URL, status, response time and, for a 5xx, the error message and stack.
3. **Find the route.** Look up the method and path in the workbook sheet APIs, or search `backend/src/modules` for the path in a `define()` call. The screen field confirms the screen.

4. **Find the service and SQL.** The handler calls a function of the module's service file; the SQL is in that function. Run the same statement in psql with the parameters from the request to reproduce.
5. **Check the history.** audit_log (by entity and entity_id) shows who changed the record and its before and after images; journals point back to their source through journal_vouchers.reference_type and reference_id.

10.4 Common failure patterns and where to look

Symptom on screen	Likely cause	Where to look
"Invalid username or password", then "Account locked"	Wrong passwords reached limits.max_login_attempts	login_history; unlock on Master > Generals > User Management
429 "Too many requests"	Sign-in or API rate limit	security.login_rate_limit, security.api_rate_limit; wait for Retry-After
"Session ended; sign in again"	Token version raised (password reset, role change, deactivation)	users.token_version; expected behaviour
403 "Requires permission: write:..."	Role lacks the permission	Master > Generals > User Management > Role Permissions
Screen missing from the menu	Role not granted the menu entry	roleMenuPermissions in utils/menuPermissions.js
"Accounting period ... is closed / soft-closed / locked"	Posting date in a closed period	Accounts > Period End > Period Management
"No active posting rule for event ..." or "GL account setting ... is not configured"	Posting configuration incomplete	Master > Finance > Posting Rules and Account Determination
"Document numbering series ... is not configured"	Numbering series missing or inactive	Master > Document Numbering
"No insurer to print: the product has no rate table in force"	Package rate table missing for the date	Master > Finance > Insurer Rate Tables
E-mails not received	notification.email_enabled off, SMTP_URL empty or SMTP error	email_outbox.status and error; Master > E-mail Outbox
A scheduled job did not run	Job disabled, scheduler off on the instance, or job error	Master > Schedules; job_runs; SCHEDULER_ENABLED
File link answers "This file link has expired"	Signed link older than FILE_URL_TTL_SECONDS	Reload the screen to get a new link
413 or 415 on upload	File too large or type not allowed	UPLOAD_MAX_MB, uploads.allowed_types
API does not start in production	Unsafe configuration	First log line names the variable (JWT_SECRET, CORS_ORIGINS ...)
/api/health answers 503	Database unreachable or migrations pending	Log lines applied <file> or migration <file> failed

10.5 Making a change safely

Add a field to a screen.

1. Add the column in a new migration (ALTER TABLE ... ADD COLUMN IF NOT EXISTS).
2. Add it to the service: the field map for inserts and updates, and the row mapper for responses.

3. Add it to the zod schema and the example in `router.js`; run `npm run export:api`.
4. Add the input to the screen, the label to `locales/en.json`, and the value to the service call.
5. Add or extend a test in `backend/test/<module>.test.js`.

Add a screen.

1. Create the component under `module/<Area>/` (back office) or `agentModule/<area>/` (operations).
2. Add the route in `routes/MainRoute.js` and the menu entry in `components/SideBar/list.js` (path and includes).
3. Grant the entry to the roles in `utils/menuPermissions.js`; extend `utils/menuPermissions.test.js`.
4. Call the API only through a service file; check that the roles hold the permissions the endpoints need.

Add a report. Add a base query in `backend/src/modules/reports/queries.js`, a `report_definitions` row in a migration or seed (code, name, category, columns, permission), and a test; see `backend/src/modules/reports/README.md`.

Add a setting. Insert the key with group, label, type and value in a seed or migration with `ON CONFLICT (key) DO NOTHING`; read it with `getSetting('group.key', <seeded value>)`; run `npm run check:settings`.

Add a migration. Create `backend/src/db/migrations/NNNN_short_name.sql` with the next number. Make it safe on a database that already has data. Never edit a migration that has been applied anywhere.

Add a scheduled job. Export a handler from `backend/src/jobs/handlers.js` and add the job to `backend/src/db/seeds/jobs.json` (code, name, cron, handler, parameters, enabled).

10.6 Release and rollback

Release (from `deploy/README.md` and `deploy/REFERENCE.md`):

1. Merge through a pull request; CI runs lint and tests on the back end and tests on the front end.
2. Take a database snapshot before a release that contains migrations.
3. Deploy the back end (EC2: `deploy/ec2/deploy.sh` installs production dependencies and reloads PM2). On start the API applies new migrations and the seed, then reports ready on `/api/health`. When a release contains migrations, start one instance first, then scale out.
4. Deploy the front end through its workflow (build, publish, cache invalidation).
5. Run the checks after deploying: `/api/health` ready, sign-in, Master > System Settings and Master > Configuration load, Master > Schedules lists the jobs.

Rollback:

- Front end: rerun the deployment workflow on the previous commit (or restore the previous published files) and invalidate the cache.

- Back end: redeploy the previous version. Migrations only add, so the previous version runs on the newer schema. Restore the database snapshot only when data written by the new version must be removed.

10.7 Coding checklist for a change

- The route is declared with `define()`, with summary, screen, example request and response.
- The route has `requirePermission()` (or is deliberately public with its own control).
- The body and query are validated with `zod`.
- SQL uses `$n` parameters only; any dynamic identifier comes from a fixed map or allow-list.
- Writes to more than one table run in `withTransaction()`.
- Every change calls `audit()`.
- Errors use the helpers of `lib/errors.js`; no detail of a server error reaches the client.
- Business values are settings read with `getSetting()`, seeded with a label; `npm run check:settings` passes.
- No secret in code or seeds; new environment variables are in `.env.example` and `deploy/REFERENCE.md`.
- No `console.log`; log through `req.log` or `logger`.
- Schema changes are in a new migration file.
- Screen texts are in `locales/en.json`; `npm run check:i18n` passes.
- The menu entry and role grants are updated; `npm run check:api` passes after `npm run export:api`.
- Tests added or updated; `npm run lint` and `npm test` pass.

11. Document Sign-Off

Serial Number	Description	Owner	Status
1	No open items at the time of issue	iorta TechNXT	

Sign Off Date	Signature	Approved By	Department & Project Role



Insurance technology for the Philippines



10+

COUNTRIES

Global Presence



8

GLOBAL OFFICES

Strategically Located



11+

MARKETS SERVED

Across Continents



100%

COMMITMENT

To Excellence



MALAYSIA



INDIA



PHILIPPINES



THAILAND



VIETNAM



USA



ETHIOPIA



CAMBODIA



HONGKONG



SINGAPORE



Scan More Details